

Advantage Shuffle

LeetCode Problem 870

Array

Greedy

Sorting

Amazon

Point72

Problem Understanding

Given two arrays **nums1** and **nums2**. If $\text{nums1}[i] > \text{nums2}[i]$, it is said to be an **advantage**.

Goal: Return any permutation of **nums1** which maximizes the advantage.

Example:

`nums1 = [2, 7, 11, 15]`

`nums2 = [1, 10, 4, 11]`

Key Insight:

Should we waste 15 to beat 1? → **No**

Should we use 2 to beat 1? → **Yes**

Rule 1: If we can beat it, beat using the *smallest possible winning number*.

Rule 2: Otherwise, sacrifice the *smallest remaining number*, saving big ones for the future.

Algorithm Steps

S1 Sort the `nums1` array.

S2

Sort the `nums2` array. However, make sure to save the original index somewhere so it will be used for updating `ans[i]`.

S3 Traverse `nums2` from largest to smallest.

S4 Check if the largest available element in `nums1` can beat the current element of `nums2`:

- **If can beat:** `ans[index] = nums1[right]` (Use the largest to beat it)
- **If can't beat:** `ans[index] = nums1[left]` (Sacrifice the smallest)

Reasoning: Because if the largest element can't beat it, no other element can. So we sacrifice the smallest element.

Implementation (C++)

```
class Solution {
public:
    vector<int> advantageCount(vector<int>& nums1, vector<int>& nums2) {
        int n = nums1.size();
        vector<int> ans(n);
        sort(nums1.begin(), nums1.end());

        vector<pair<int, int>> arr;
        for(int i = 0; i < n; i++) {
            arr.push_back({nums2[i], i}); // index chahiye answer array
            // update krne ke liye maan lo pata lag gya ki nums1 ka falana element nums2
            // ko sabse closely beat krta he but usko place kaha krenge ? uske liye i
            // chahiye
        }

        sort(arr.begin(), arr.end());

        int l =
0; // agar beat nhi kr paaye to smallest discard krenge nums1 se
        int r = n - 1; // highest ko beat krne ke liye highest

        for(int i = n - 1; i >= 0; i--) {
            int val = arr[i].first;
            int idx = arr[i].second;

            if(val < nums1[r]) {
                ans[idx] = nums1[r];
                r--;
            }
            else {
                ans[idx] = nums1[l];
                l++;
            }
        }

        return ans;
    }
};
```

